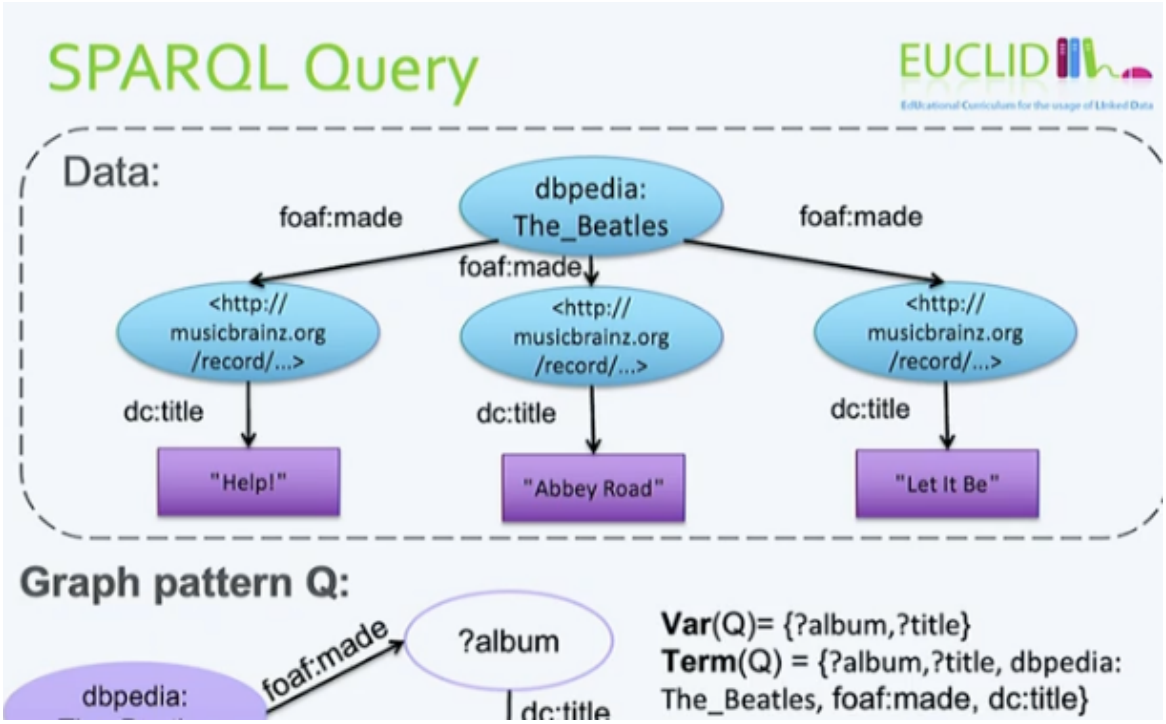
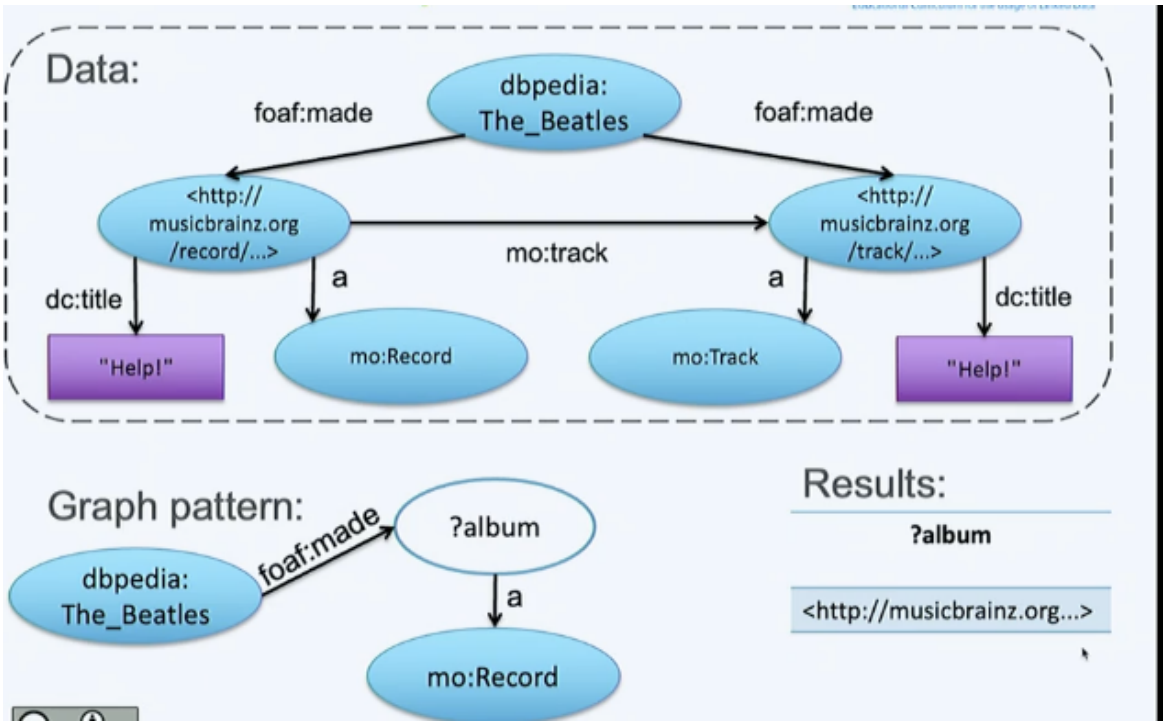
- E' un linguaggio di interrogazione per gradi RDF pubblicati sul web
  - Quindi si basa sull'HTTP, con client e server
  - Quindi è diviso in vari protocolli
    - Protocollo client, che fa richieste per fare query spartql
    - Protcollo servizio, che gestisce le richieste per le operazioni
    - Protocollo endpoint, che ascolta le richieste del client (è in mezzo tra client e service)
  - Fa parte del semantic web stack
    - Aka, il web deve essere fatto da standard aperti, e costruirci sopra
    - E poi permettere ad altre persone di accedere
    - Secondo questo paradigma, dobbiamo definire la terminologia comprensibile alla macchina
- Com'è fatto:
  - Triple, sono building block
    - Un soggetto o iri o blank node (no literal)
    - Il predicato deve essere un iri
    - L'oggetto un iri o blank node o literale
  - Var(Q) sono tutte le variabili
  - Term(Q) è l'unione delle variabili, nodi e tuple
  - 

  - RDF triple pattern
    - E' una tripla in cui posso usare in qualunque punto una variabile

- Le variabili si esplicitano con un punto interrogativo davanti
    - dbpedia:The_Beatles foaf:made ?album.
      ?album mo:track ?track.
- Patter matching
    - Le query descrivono sottografi, e stiamo cercando patterns specificato con ? o $
    - 
      
- Una valutazione è quando riusciamo a sostituire successivamente da un insieme di variabili e delle costanti
    - Una funzione mapping usa la valutazione per sostituire parti
        - ?album -> <...help!> ?title -> Help
          Qui stiamo cercando l'album help! e stiamo sostituendo il titolo con Help
    - L'evalutazione viene fatto anche senza sostituire letteralmente, ma per fare le ricerche
    - Problema: la valutazione viene fatto in maniera ricorsiva
        - E anche che ci potrebbero essere facilmente dei duplicati

- Esempio query
  ```sparql
  prefix dbpedie:
  SELECT ?album
  FROM => l'uri del grafo che vogliamo prendere
  - Non è obbligatoria la from
  WHERE {
  - La clausola where ci dice come manipolare i dati per prendere i dati, aka gli if
  dbpedia:The_Beatles foaf:made ?album.
  - Seleziono album, dove i beatles (dbpedia: The_Beatles) hanno fatto (foaf:made) un album
  ```

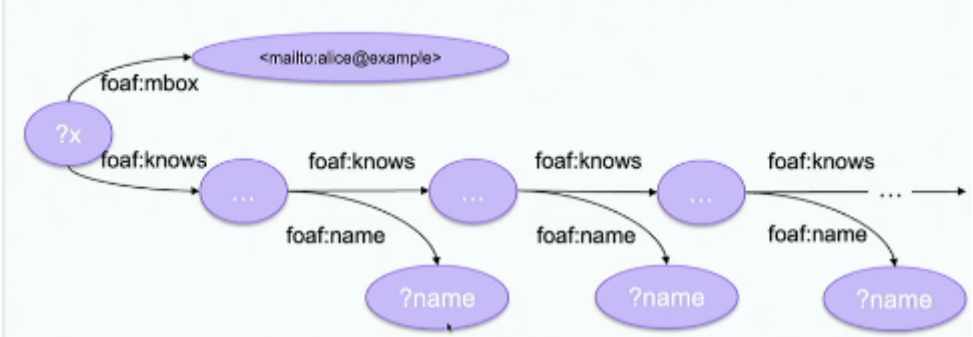
(?album)
?album a mo:Record ; dc:title ?title
- Questo album (?album) è un record (mo:Record) ed ha un titolo (; dc:title ?title)
}
ORDER BY ?title

- Queries forms
  - select (ritorna le variabili, proiezione)

```
SELECT * WHERE {
  ?s ?p ?o .
}
```

  - Simile al sql
  - Abbiamo anche altre opzioni di selezione:
    - Ask, si/no
    - Describe, Ritorna grafo rdf del grafo riguardante le risorse
    - Construct where, ritorna grafo rimodellandoli come noi vogliamo, riscrivendolo
- Path expressione
  - Si attraversa il grafo
- Path queries
  Queries particolari, estendono il tutto, qui le variabili possono apparire solo nell'oggetto e nel soggetto, mai predicato
  Capita in esame: qui le variabili **non** vengono usate nei predicati
  Non sono path expression, ma sono particolari dove le variabili possono essere usati come nodi
  I risultati di una path queries sono **sempre** coppie di nodi (una tabella)
  Noi qui attraversiamo coppie di nodi
  Noi qui possiamo usare variabili nel soggetto
  La valutaizone delle path queries **sono sempre e solo** coppie di nodi
  - or: |
    - Matchare una delle due possibilità {:book1 dc:title|rdfs:label ?displayString}
  - Sequenza da seguire: / (io lo considero come e, and)
    - {?x foaf:knows/foaf:knows/foaf:name ?name}
      Trova tutti i nomi che sono 2 "foaf:knows" di distanza
  - Inversione: ^
    - {?x foaf:mbox <alice>} = {<alice> ^foaf:mbox ?x}
    - Sono la stessa cosa, questo può essere combinato per fare tipo
      {?x foaf:knows/^foaf:knows ?y FILTER(?x!=?y)}
      Trova un x (?x) che conosce qualcuno (foaf:knows) and (/) che conosce x (siccome c'è ^, allora torniamo indietro verso x)

- Un numero arbitrario di qualcosa: +




*Arbitrary length match*: Find the names of all the people that can be reached from Alice by `foaf:knows`:

```
{
  ?x foaf:mbox <mailto:alice@example> .
  ?x foaf:knows+/foaf:name ?name .
}
```

61

-